

SpikeGPT Reimplementation Final Report
Julian Bushlow, Kathy Chen, John Palsberg, Angelina Zhou
Github: <https://github.com/jbushlow/spiking-transformer-cs4782>

Introduction

Transformers are an incredibly powerful and applicable AI architecture for modern workloads. However, training and running traditional architectures, such as a GPT, at scale is highly energy-expensive, which results in high computational costs for LLM providers as well as raises environmental concerns. Spiking neural networks (SNNs), which are based on biologically-inspired “spiking neurons,” have the potential to reduce the high computational costs of LLM training and inference. Firstly, they replace the expensive matrix multiplications which pervade traditional neural networks with much cheaper addition-accumulate operations. Secondly, they reduce the total number of computations during operations as spiking neurons only require computations when they output information, which is only a fraction of the time. These factors can make spiking LLMs over an order of magnitude more efficient than traditional transformers.

The paper we aimed to reproduce is “SpikeGPT: Generative Pre-trained Language Model with Spiking Neural Networks,” by R. Zhu, Q. Zhao, G. Li, and J. Eshraghian.” The paper builds a modified transformer architecture based on the known SNN architectural framework by replacing certain transformer sub-blocks with blocks more suited for efficient SNN implementation and operation, notably replacing self-attention with a recurrence-based mechanism. Ultimately, the paper manages to capture the energy-efficiency benefits of SNNs within a larger LLM architecture.

Methodology

SpikingNeuron Background

The SpikeGPT architecture is based around the Leaky Integrate-and-Fire (LIF) abstract neuron model, which models three key aspects of biological neuron function in a discretized form. Biological neurons send information in voltage spikes, which can be represented as binary 0s and 1s. These spike inputs change the internal state of a neuron, adding (integrating) a set weight each time a spike happens. Between time steps, the neuron’s state “leaks,” exponentially falling back to 0. Finally, if the internal state passes a certain threshold, the neuron fires a spike and resets its state. All of these steps are implemented in a discrete difference equation shown in Figure 1, where β is a constant encoding the exponential leak, T is the threshold, and $S[t]$ is a binary flag which equals 1 when U (the internal neuron “voltage”) goes above the threshold. Note that despite $WX[t]$ being a standard vector product, the inputs being binary reduces the multiplication to a conditional add, removing the need for a multiplier. An example of these neuron dynamics is shown in Figure 2.

Architecture

We reimplemented the following SpikeGPT architecture (Figure 3) from the paper. The core architecture consists of a BinaryEmbedding Layer, which maps each input token into a binary n_embd dimensional vector. This is done through a normal embedding layer, followed by an arctangent surrogate function that binarizes the vector, converting positive floating point values to 1 and negative floating point values to a 0. After the embedding layer, we feed the input through many stacked spike blocks. These blocks consist of these layers in order: layer normalization, SpikingRWKV, layer normalization, Spiking RFFN, and dropout. The layer normalization and dropout is similar to that in standard transformers. Spiking RWKV replaces standard self-attention with a $O(n)$ recurrent mechanism. It first blends each token’s representation with the previous token’s representation through learnable mixing coefficients, allowing the model to look back one time step. Then we feed the blended input into the Weighted Key Value (WKV) recurrence, which is a weighted average of all past values. Each value’s weight is determined by a key and additionally decays exponentially through a learned time-decay parameter. Spiking RWKV allows the model to learn context recurrently without using pairwise interactions like in standard transformers. Spiking RFFN replaces the feed forward layers in standard transformers. This is similar to a normal feed forward layer with spiking neurons except we used squared ReLU as the paper found it performed better for sparse spiking activations. After the n spiking blocks, we finally have a linear layer to map from the embedding space to 256, all possible byte values. The hyperparameters n_embd and n are 512 and 12 to match the paper’s configuration for their 46 million parameter model. While the paper trained both the 46 million and 216 million parameter models, we only trained the 46 million parameter model due to limited compute resources.

Training and Datasets

We trained our model on the Enwik8 language modeling dataset with a next-character prediction objective. We did next-character prediction because the dataset is byte-level. We used Adam as our optimizer, cosine decay for our learning schedule, and gradient clipping of norm 1.0. We pre-trained with this setup for 202 epochs and evaluated on natural language generation (NLG) tasks. We also fine-tune the model for natural language understanding tasks (NLU). We fine-tuned on SST-2, SST-5, MR, and Subj datasets consisting of positive vs. negative sentiment analysis tasks and objectivity vs. subjectivity tasks.

Evaluation Metrics

For NLG tasks, we used Bits per Character (BPC) as our evaluation metric. It measures the average number of characters required to encode a character representing the average "surprise" or uncertainty of the model. A lower BPC means better next character prediction. For NLU tasks, we used accuracy for our evaluation.

Extensions (Modifications to Original Methodology)

The paper uses a fixed $\beta = 0.5$ where β is a hyperparameter controlling how much the neuron remembers across time steps (see equation in figure 1). We modified β to be a learnable parameter. We started using a learnable β from different epochs in the training, comparing it to the validation loss of the model being trained with a fixed β from the same epoch.

Chosen Results and Analysis

We evaluated our model on both NLG and NLU tasks. Figure 4 shows the results of our model on NLU tasks compared with the paper and BERT. We achieved similar results to the paper on all benchmarks except for the MR movie review dataset where our model performed around 20% worse than both other models. Both the paper's model and our model performed worse than BERT by 10-20%, indicating that SpikeGPT functions less well because of its recurrent model which stores keys and values in hidden states, but well enough to make the model viable.

This reduction in accuracy is balanced by massive improvements in energy efficiency. While the base GPT model requires every multiplication in the self attention and feed forward layers to happen every single input (using around 4.5 pJ per MACop), our model only uses accumulates, which use 5x less energy and also only happen when a neuron fires, massively reducing the amount of energy used. Figure 5 shows the energy used per layer, with rates much lower than 100%. Using high-level energy calculations similar to the paper's, we find that our SpikeGPT architecture uses 36.2x less energy as a standard GPT.

We also evaluated the model on generation tasks (NLG), results shown in Figure 6. Our model performed worse than both the paper and BERT on all datasets since it had a significantly higher BPC, though better than a stacked LSTM model. This indicates that our model could have benefitted from more training or fine-tuning with different hyperparameters. The authors could have cherry-picked their results or left out details needed for reimplementing.

The results of our extension on the original paper are shown in Figure 7. We did not see much improvement when we made β a learnable at a later stage in training (at epoch 197), but when we used a learnable β from an earlier epoch (25), we saw a noticeable improvement in training loss drop. Values of β changed relatively slowly, with the maximum spread of β values increasing by around .01 every 10 epochs. Thus, we expect that training for the full 200 epochs would lead to a noticeable increase in model performance, making this a potential topic for future study.

Overall, our results mostly match those of the paper, with SpikeGPT achieving good results on both language generation and understanding tasks. Performance is worse than that of standard GPTs, but the massive decreases in energy make the cost worthwhile. The power efficiency and better inference time complexity open the door for embedded deployment, which could significantly help robots which need to make decisions while running with strict constraints.

Reflections

Training our model from scratch required an immense amount of time and careful selection of hyperparameters. This project taught us the amount of time that goes into building a working model and that each epoch presents a new set of choices. We learned the stress that comes from wondering whether to stop training now and start finetuning, or keep training for another few hours to lower the loss a little more. Most humorously, this project taught us the limits of Google Colab, as we had to deal with checkpointing, runs crashing, and Google Drive storage issues (since checkpoints quickly fill the 15 GB limit).

References

- [1] Zhu, R. et al. (2022). SpikeGPT: Generative Pre-trained Language Model with Spiking Neural Networks. Transactions on Machine Learning Research (TMLR). <https://arxiv.org/abs/2302.13939>
- [2] Cornell University Department of Computer Science. Weinberger, K. et al. CS 4782 Deep Learning. www.cs.cornell.edu/courses/cs4782/2026sp/

$$U[t] = \beta U[t - 1] + W X[t] - \theta S_{out}[t]$$

Figure 1: Spiking Discrete Difference Equation. Governs LIF neuron dynamics.

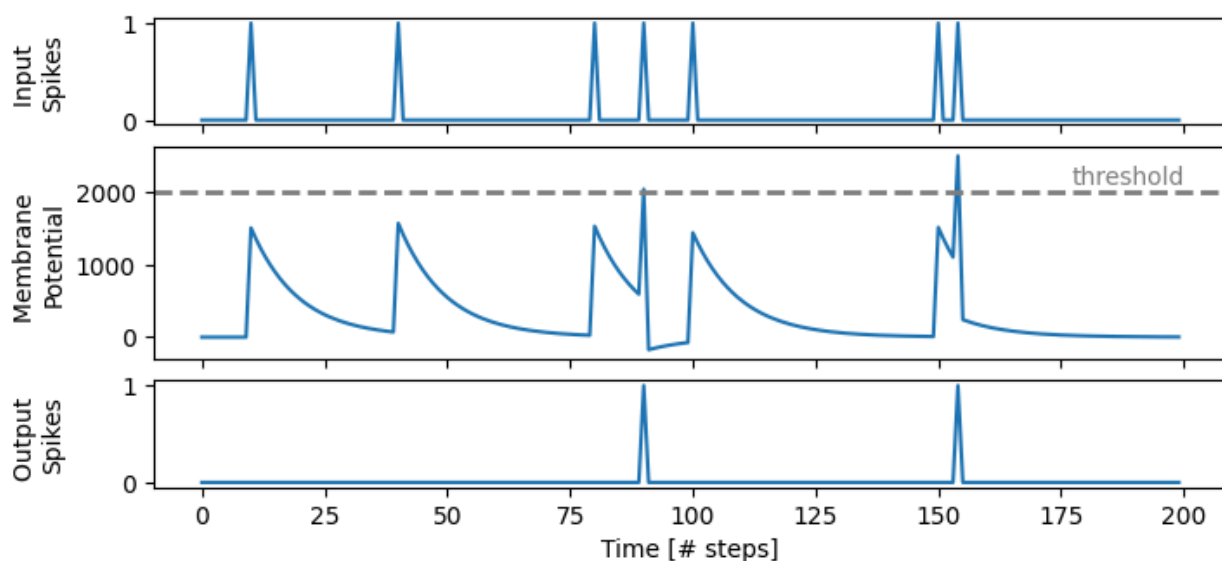


Figure 2: Example Spiking Neuron Inputs, States, and Outputs

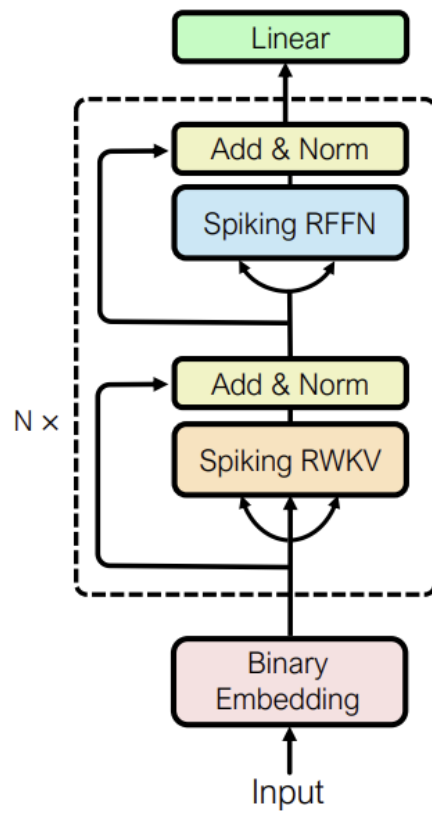


Figure 3: SpikeGPT Transformer Architecture

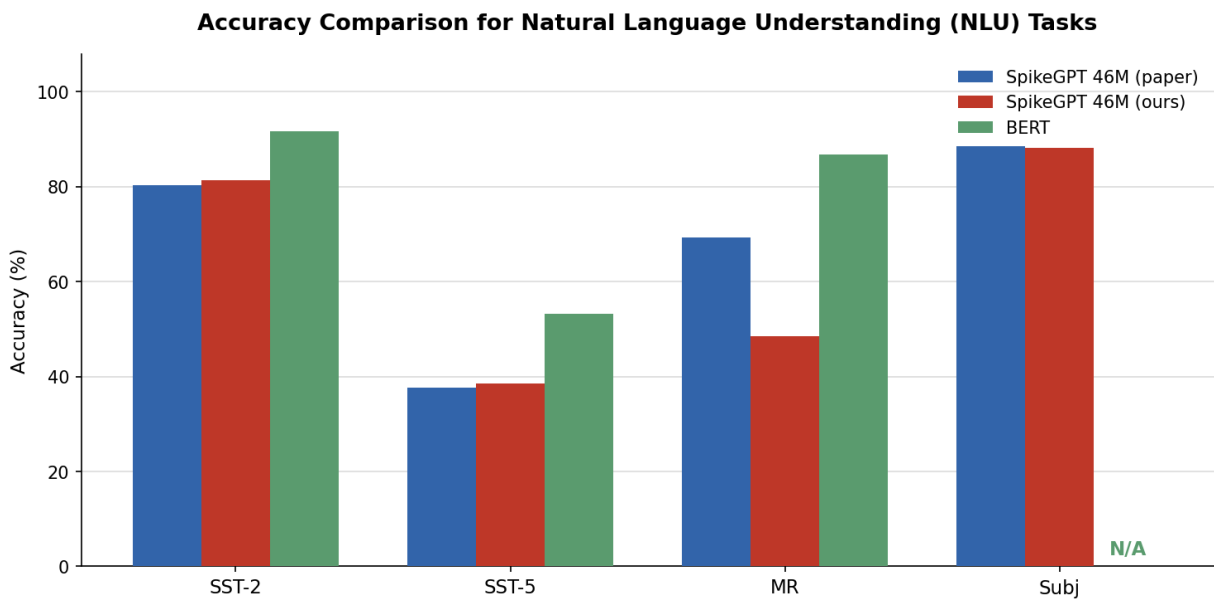


Figure 4: Accuracy Results and Comparison for NLU Tasks

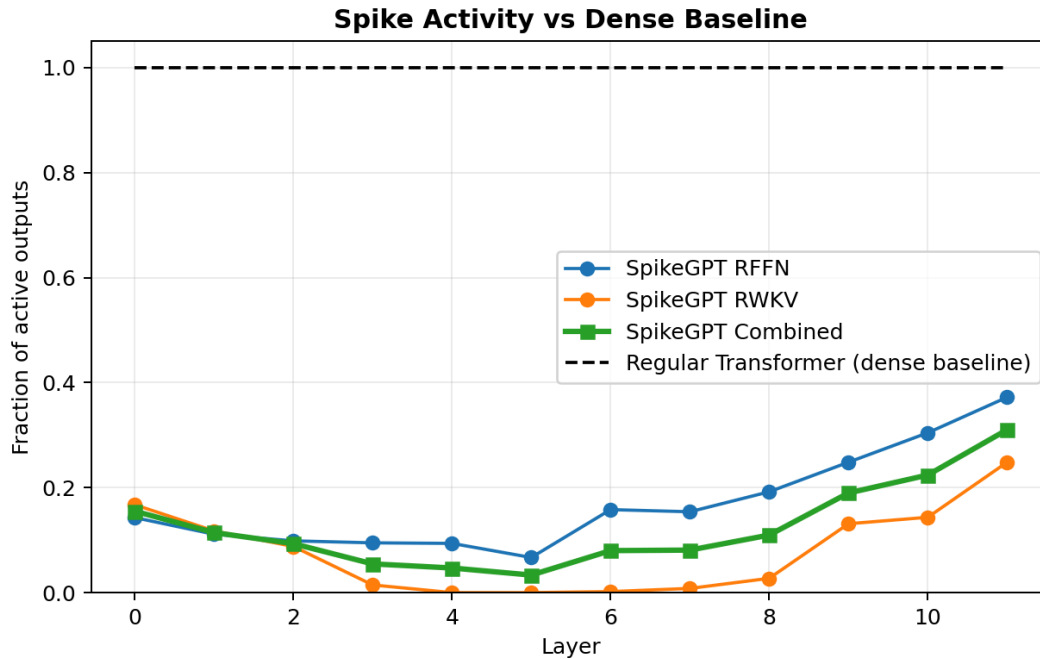


Figure 5: Transformer Neuron Activation () Density Comparison

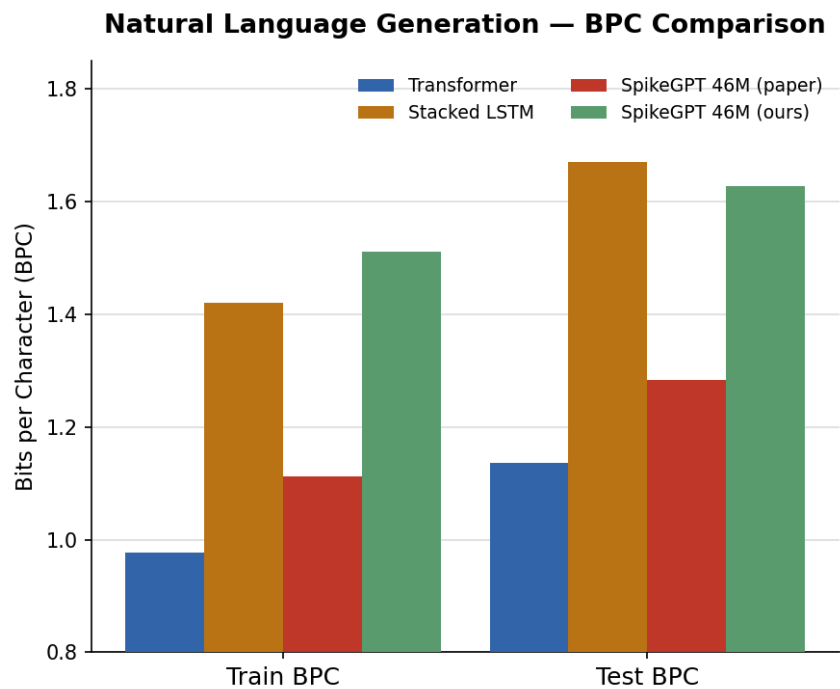


Figure 6: BPC Comparison for Natural Language Generation across architectures

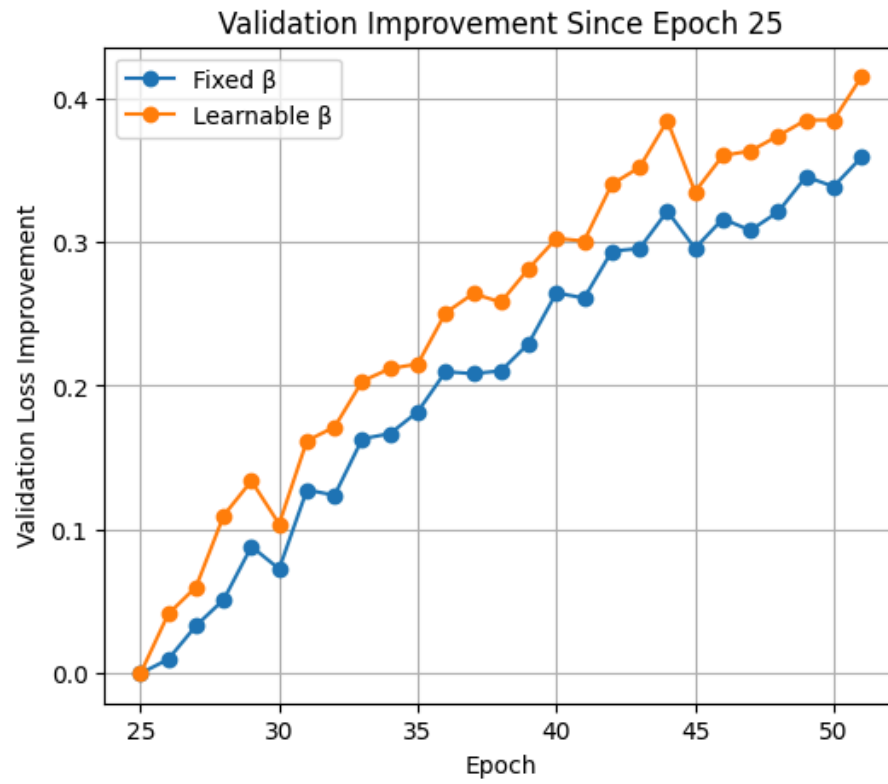


Figure 7: Validation Loss Improvement After Making β a Learnable Parameter Starting From Epoch 25